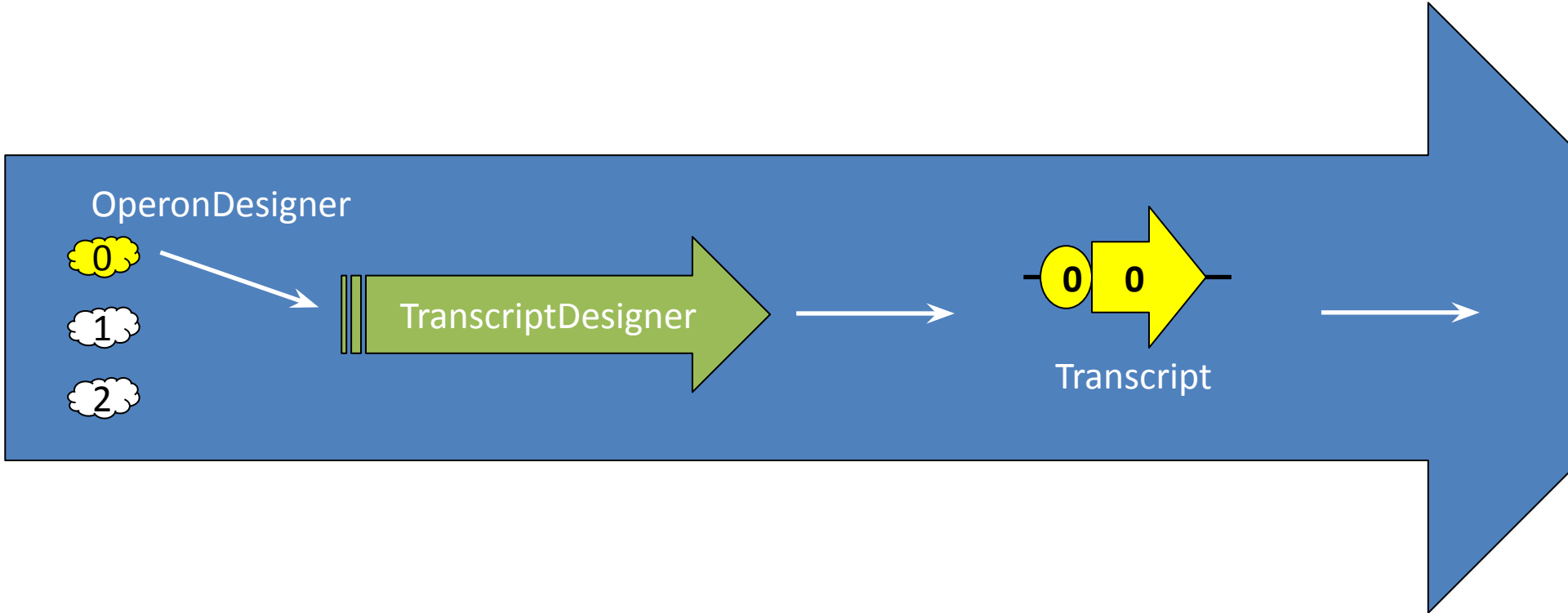
Gene Design

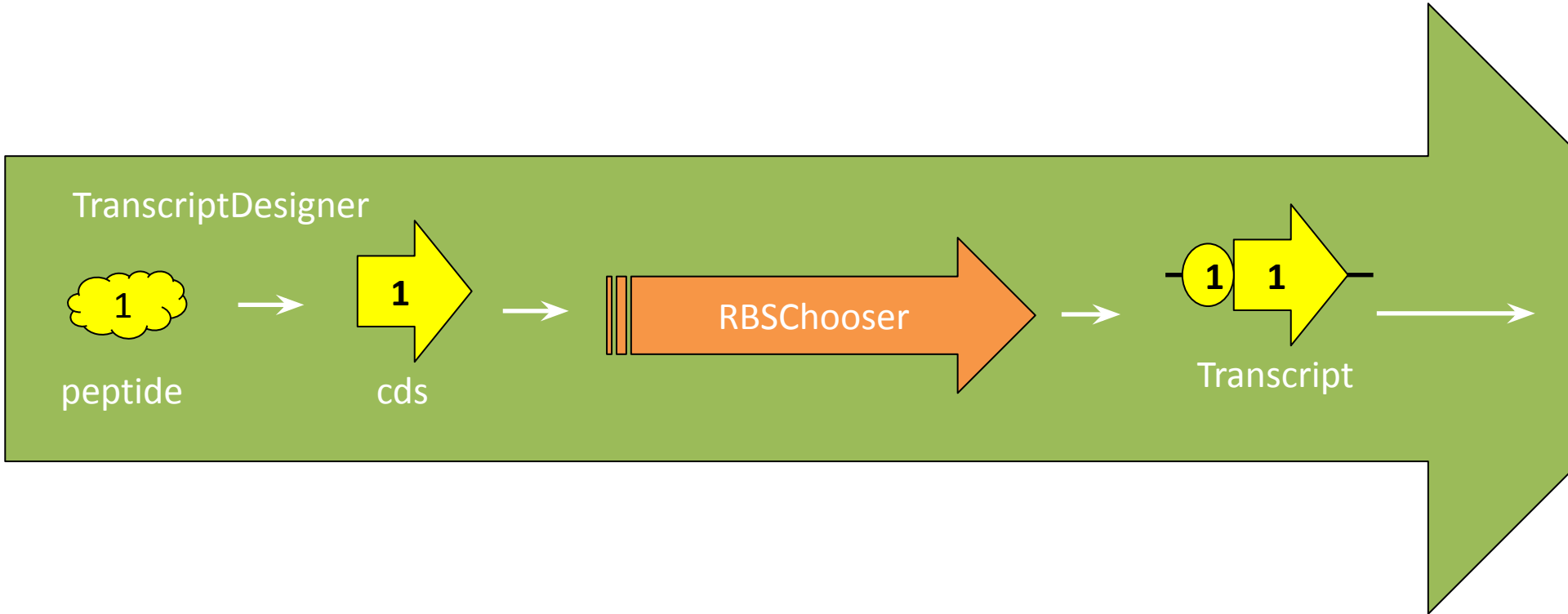
Why Codon Optimize

- Proteins-to-DNA is not a deterministic problem: there are many equally valid solutions. Must pick something.
- Need to avoid special sequences that must be preserved as absent to enable fabrication (BsaI, BsmBI, AarI)
- Must avoid homopolymeric runs
- Avoid 20+ bp regions of excessively low or high GC content
- Avoid repetitive sequence that can recombine
- Avoid internal start codons/ribosome binding sites and promoters
- Avoid premature termination and RNAse cleavage
- Want robust CDS expression, even if need overall low expression

OperonDesigner



TranscriptDesigner



Transcript

```python
from dataclasses import dataclass
from typing import List
from .rbs_option import RBSOption  # Assuming RBSOption is defined in rbs_option.py

@dataclass(frozen=True)
class Transcript:
    """
    Encodes a monocistronic mRNA from an RBS and a coding sequence.
    """
    rbs: RBSOption
    peptide: str
    codons: List[str]
```

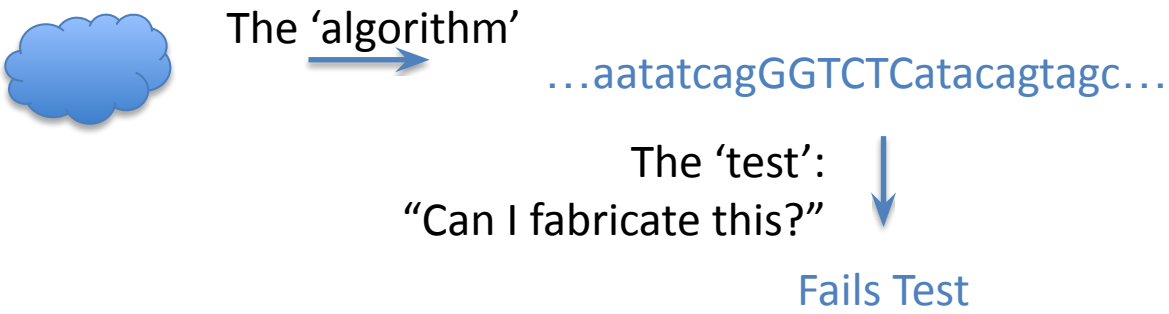
Operon

```python
from dataclasses import dataclass
from typing import List
from .transcript import Transcript  # Assuming Transcript is defined in transcript.py

@dataclass(frozen=True)
class Operon:
    """
    Encodes a genetic construct described in terms of a single operon
    encoding multiple cocistronic transcripts.
    """
    transcripts: List[Transcript]
    promoter: str
    terminator: str
```

Checkers

(a.k.a Validation Functions and related types)

Domain-specific constraints



- The issue: both the algorithm and the test need to know about GGTCTC being a problem for fabrication
- It requires that the test and algorithm be identical in their logic, resulting in duplication of code
- The Checker is to centralize this logic for use by both the algorithm and the test

Checker Functions in src

- Like ForbiddenSequenceChecker, InternalRBSChecker, GCChecker
- A Function that Inputs a design, examines with domain-specific logic, and return true, false, or throws an Exception
- May be invoked inside algorithms to select amongst a bag of options
- Is invoked by JUnit tests paired with edge case arguments (ie, MKKKKKK)
- Is invoked by benchmarks on a large bag of random input arguments
- Would also have tests of the Checker's behavior. Ie, SequenceChecker should return true for AAAAGGTCTCAAAAA, false on ACACAC, and throw an Exception on BOB
- The highest level design Function invokes the Checkers before returning or persisting the result, and if fails raise an Error

ForbiddenSequenceChecker

- A Function for evaluating the presence of forbidden sequences. Hard codes restriction sites to conserve

```python
class ForbiddenSequenceChecker:
    def __init__(self):
        self.forbidden = []

    def initiate(self):
        # Populate forbidden sequences
        self.forbidden = [
            "AAAAAAAA",  # poly(A)
            "TTTTTTTT",  # poly(T)
            "CCCCCCCC",  # poly(C)
            "GGGGGGGG",  # poly(G)
            "ATATATAT",  # poly(AT)
            "CAATTG",    # MfeI
            "GAATTC",    # EcoRI
            "GGATCC",    # BamHI
            "AGATCT",    # BglII
            "ACTAGT",    # SpeI
            "TCTAGA",    # XbaI
            "GGTCTC",    # BsaI
            "CGTCTC",    # BsmBI
            "CACCTGC",   # AarI
            "CTGCAG",    # PstI
            "CTCGAG",    # XhoI
            "GCGGCCGC",  # NotI
            "AAGCTT",    # HindIII
        ]
```

ForbiddenSequenceChecker

- A Function for evaluating the presence of forbidden sequences. Hard codes restriction sites to conserve

```
def run(self, dnaseq):
    # Use the reverse_complement function from seq_utils
    rc = reverse_complement(dnaseq)
    combined = (dnaseq + "x" + rc).upper()

    for site in self.forbidden:
        if site in combined:
            return False, site

    return True, None
```

test_forbidden_sequence_checker.py

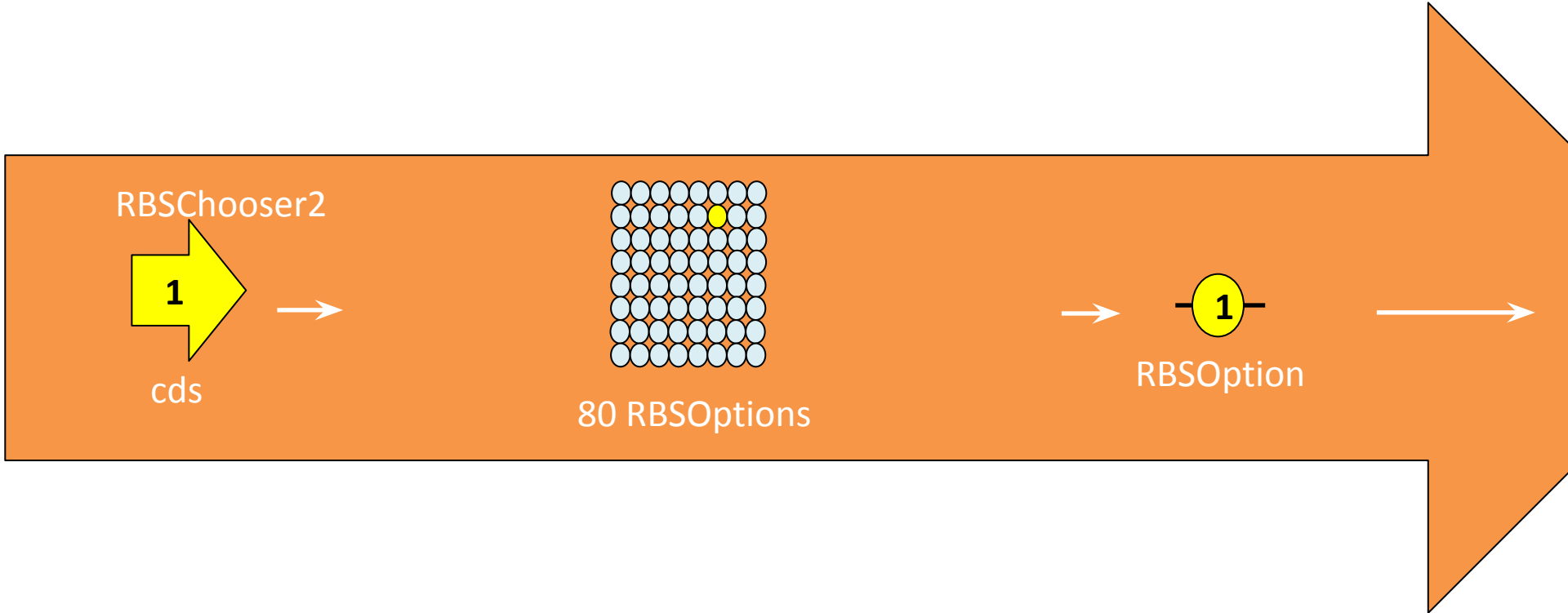
```python
import pytest
from genedesign.checkers.forbidden_sequence_checker import ForbiddenSequenceChecker

@pytest.fixture
def checker():
    checker = ForbiddenSequenceChecker()
    checker.initiate()
    return checker

def test_forbidden_sites(checker):
    # Sequences with forbidden sites
    forbidden_seqs = [
        "TTGACAATTgaattcCGAACTAGTATAAT",
        "TTGACACGGTCTCATTCACTAGGTTATAAT",
        "TTGACAGTCTAGAGTCTGAACAAGGAGATCTTAAT",
    ]

    print(">> Testing problematic sequences (expected False)")
    for seq in forbidden_seqs:
        result, site = checker.run(seq.upper())
        print(f"result: {result} on {seq}")
        assert result == False

def test_allowed_sites(checker):
    # Sequences without forbidden sites
    allowed_seqs = [
        "AAACTGTAATCCACCACAAGTCAAGCCAT",
        "GTTCCGCTGGCGATCCATCGTTGGCGGCCG",
    ]
```

Where to encode

- All Functions are in src, and Checkers are Functions
- All Checker Tests are PyTests, all PyTests are in tests
- Package structure is mirrored
- Checkers are segregated in their own sub-package in src, as was done with data and Models
- Test names are automatic for AlgorithmTest; e.g. gc_checker.py and test_gc_checker.py

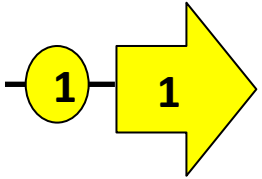
Algorithm

- The Function that you are testing; for example RBSChooser:



Checker

- A Function that inputs potential solutions and returns true/false, eg ForbiddenSequenceChecker



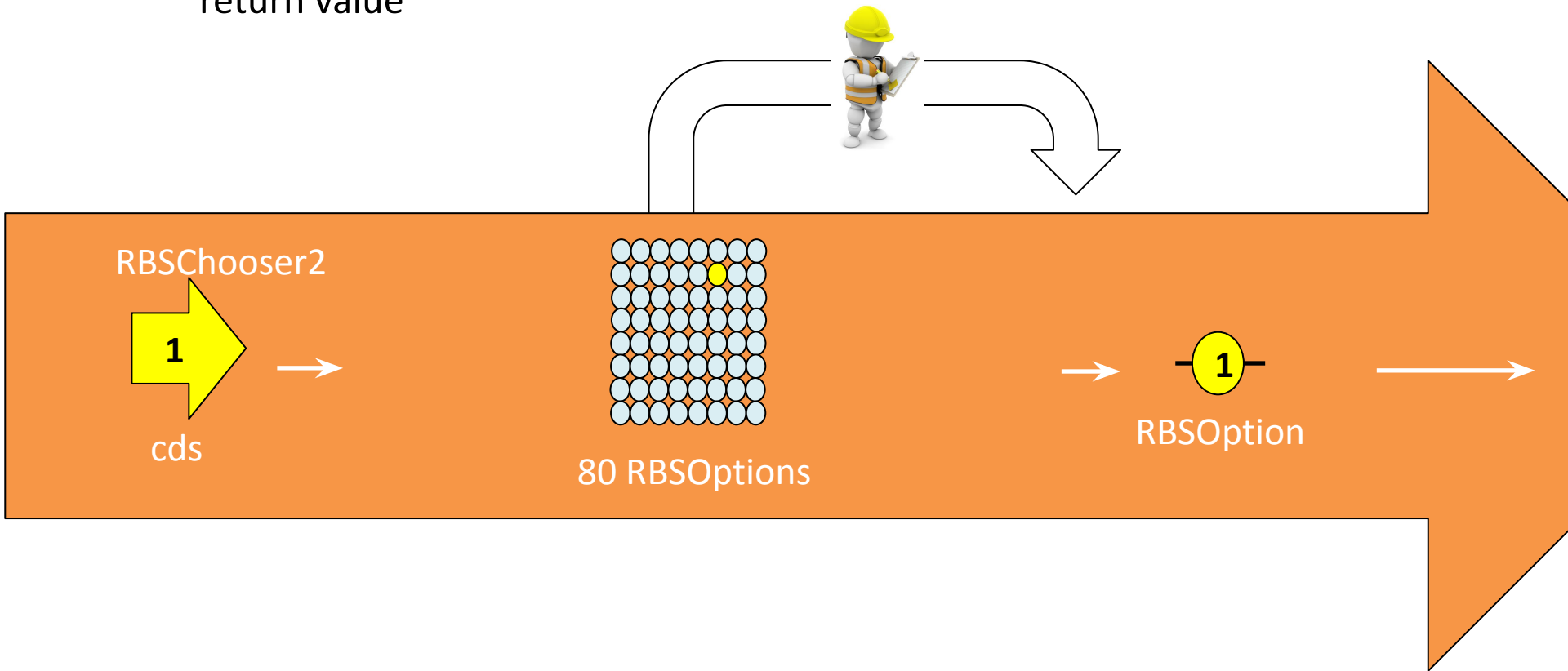
GGATCC
AGATCT
GGTCTC

true/false

AAAAAAAA
TTTTTTTT
CCCCCCCC
GGGGGGGG

Invocation in Algorithm

- The Checker is invoked within the Algorithm to guide decisions about the return value



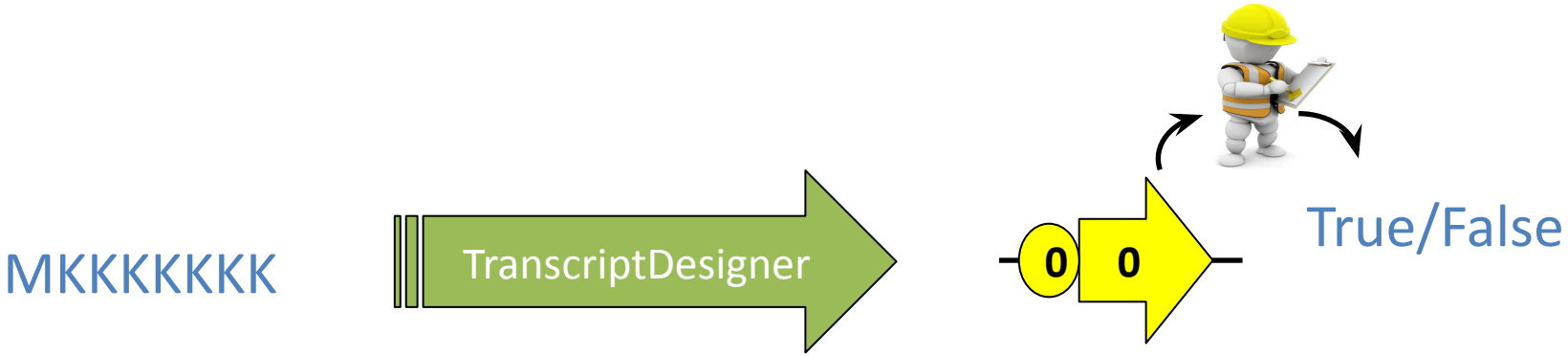
CheckerTest

- A PyTest that asserts domain logic on the Checker, eg test_forbidden_sequence_checker
- Goes in test, in a mirrored checkers package

//A few assertions about behavior on OK sequences	
ACACACACAC	FALSE
AAAAAA	FALSE
//An assertion for each forbidden sequence	
AAAGGATCCAAA	TRUE
AAAAGATCTAAA	TRUE
AAAGGTCTCAAA	TRUE
AAAAAAAAAAAAAA	TRUE
AAATTTTTTTTAAA	TRUE
AAACCCCCCCCAAA	TRUE
AAAGGGGGGGGAAA	TRUE
//Assertions on syntactic variation	
aaaaaaaaaaaaa	TRUE
aaaGGATCCaaaa	TRUE
aaaaaXaaaaaaa	exception
aaaaaa aaaaaa	exception

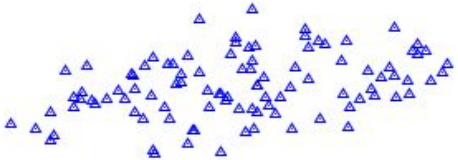
Edge Case Test

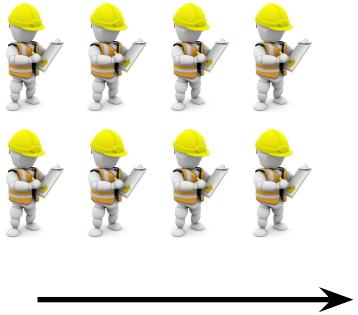
- A PyTest that examines the Algorithm on specific inputs that are prone to causing incorrect behavior
- In the case of TranscriptDesigner, the MKKKKKKK peptide is a reliable challenge because it is prone to causing poly-A solutions which are forbidden
- Goes in test, mirroring package location of Algorithm class

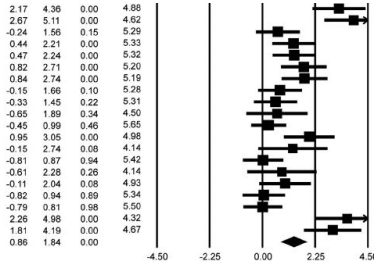


Invocation in Benchmark

- A benchmarking class that passes many CDS through the Algorithm and collects statistics about its performance
- That performance is evaluated using all the Checkers
- It goes in the test code in a benchmarking package

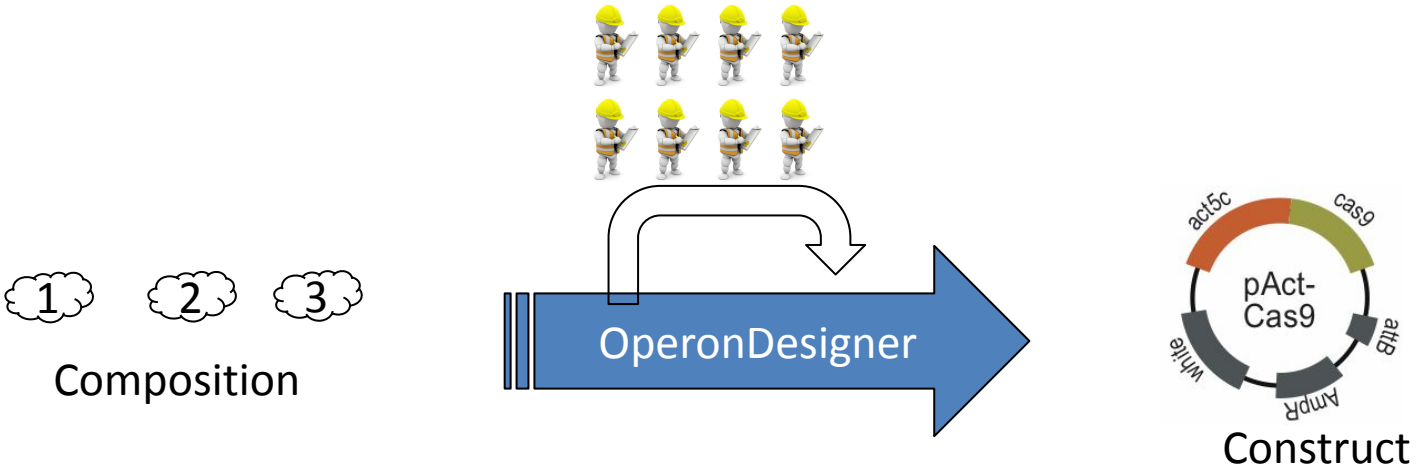






Invocation in highest-level Function

- Before returning its design, the highest-level Function could use all the Checkers to make one final check of the data before returning
- Goes in the high-level Function, in src
- Part of the runtime of the program, not the tests
- For this project, the highest-level Function is OperonDesigner



Which Checkers

Which of these Checkers should not be used to evaluate the validity of a Transcript (DNA with RBS and CDS)?

A) ForbiddenSequenceChecker
B) SecondaryStructureChecker
C) InternalRBSChecker
D) InternalPromoterChecker
E) CAIChecker
F) RepeatSequenceChecker

TranscriptDesigner

One AA one Codon

```python
class TranscriptDesigner:

    def initiate(self) -> None:
        self.rbsChooser = RBSChooser()
        self.rbsChooser.initiate()

        # Codon table with highest CAI codon for each amino acid (for E. coli)
        self.aminoAcidToCodon = {
            'A': "GCG", 'C': "TGC", 'D': "GAT", 'E': "GAA", 'F': "TTC",
            'G': "GGT", 'H': "CAC", 'I': "ATC", 'K': "AAA", 'L': "CTG",
            'M': "ATG", 'N': "AAC", 'P': "CCG", 'Q': "CAG", 'R': "CGT",
            'S': "TCT", 'T': "ACC", 'V': "GTT", 'W': "TGG", 'Y': "TAC"
        }

    def run(self, peptide: str, ignores: set) -> Transcript:
        # Translate peptide to codons
        codons = [self.aminoAcidToCodon[aa] for aa in peptide]

        # Append the stop codon (TAA in this case)
        codons.append("TAA")

        # Build the CDS from the codons
        cds = ''.join(codons)

        # Choose an RBS
        selectedRBS = self.rbsChooser.run(cds, ignores)

        # Return the Transcript object
        return Transcript(selectedRBS, peptide, codons)
```

https://classroom.github.com/a/VDGHGHRb

Project 3: TranscriptDesigner

Write:

- An improved implementation of TranscriptDesigner that generates designs that satisfy low hairpin count, high CAI, and the absence of internal promoters and forbidden sequences
- A new test or benchmark. There should be a pytest involved somewhere, and most likely you will choose to write a Checker and CheckerTest. You should include your new Checker in your TranscriptDesigner
- Optional extension: make a multi-operon designer function

Your code should conserve the original Model scripts. The correctness will be evaluated with proteome_benchmarker.py and review of your code.

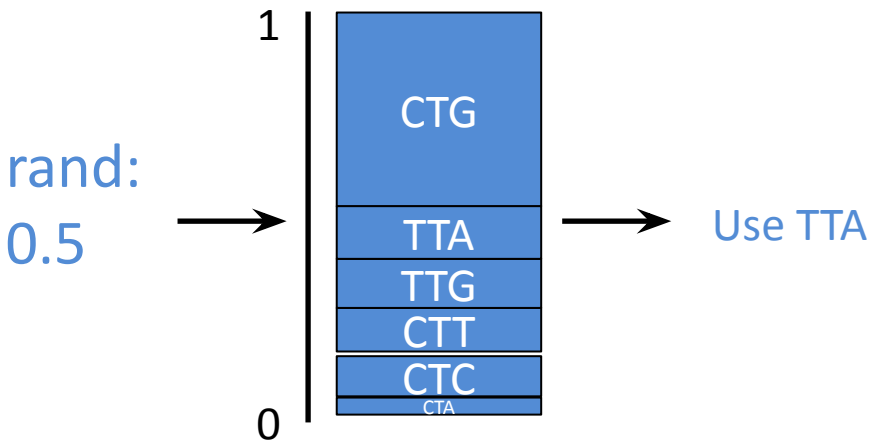
What you turn in is the URL for your repo.

‘Guided random’ method

- “a Monte Carlo algorithm that selects codons at random based on the frequencies of use of each codon in the reference set”
- Can iteratively generate random solutions until one satisfies the testing criteria (ie, no forbidden sequences, no secondary structure, etc.)
- Could also iterate a large number of random solutions, score and rank the valid solutions

Leucine

	RSCU
CTG	0.49
TTA	0.13
TTG	0.13
CTT	0.11
CTC	0.10
CTA	0.04



‘Guided random’ method

ATG	AAA	AAA	AAA	AAA	AGC	TAC	GGG	GTC	CAA
M	K	K	K	K	S	Y	G	V	Q

poly(A) detected

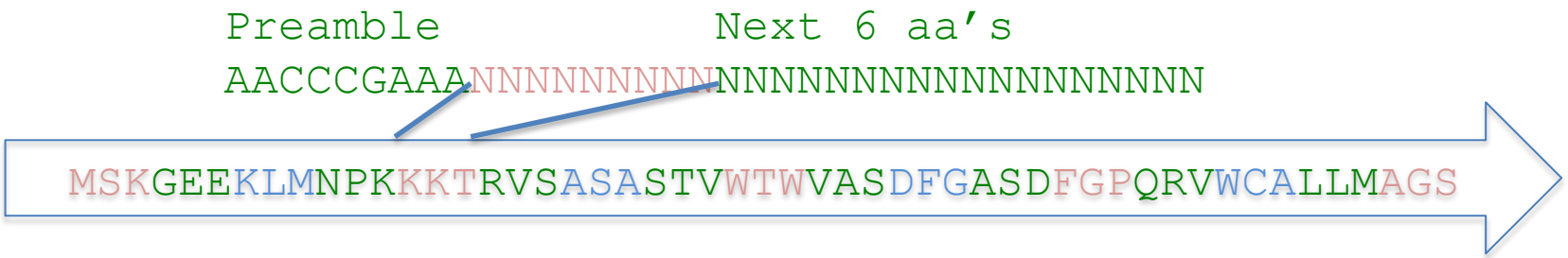
Sliding Window

1. Break the sequence into 3aa windows (9 nt each), scan through them left to right and optimize them individually

2. Enumerate sliding windows of each 9bp oligonucleotide in combination with its upstream (already chosen, preamble) and downstream (undetermined, next 6 aa's) oligonucleotides

3. Rank each window by forbidden sequence, secondary structure, etc. Choose one

4. Retain the middle 3 codons (toss out the next ones) and move on to the next window



PMID: 21189842

Sliding Window

ATG AAA GTC CAG GGC TAT ATT TTG CCC

M K V Q G Y I L P S T G F L R Q H Y T Q R

proteome_benchmarker.py

- Parses genome from FASTA file.
- Designs transcripts with **TranscriptDesigner**.
- Analyzes errors and validates transcripts.
- Generates reports on success, errors, and validation.

Validation Checks:

1. **Translation Match** – Transcript correctly translates to original protein.
2. **Hairpin Detection** – Identifies harmful secondary RNA structures.
3. **Forbidden Sequences** – Flags unwanted sequences like restriction sites.
4. **Internal Promoters** – Detects unintended promoter sequences.
5. **Codon Usage** – Validates codon diversity and checks for rare codons.

summary_report.txt

Total genes processed: 2458
Parsing runtime: 0.05 seconds
Execution runtime: 11.32 seconds
Total exceptions: 0
No exceptions encountered.

Total validation failures: 6956

Validation Failures by Checker:
- Forbidden Sequence Checker: 1924 occurrences
- Hairpin Checker: 2326 occurrences
- Codon Usage Checker: 2322 occurrences
- Promoter Checker: 254 occurrences
- Translation/Completeness Checker: 130 occurrences

Your score for the TranscriptDesigner part of the project is based on this report that will be generated from a different genome, and code review

Test component

1. Implement one of RNAseE_Checker, Internal_RBS_Checker, or RNA_Interference_Checker
2. …or come up with your own novel Test, Benchmark, or Checker/CheckerTest
3. Implement a pytest either as a test of your Checker, or a direct test of TranscriptDesigner
4. Incorporate your Checker into the implementation of TranscriptDesigner

RNaseE_Checker

Identifies potential RNase E cleavage sites within the designed RNA, which could lead to unintended RNA degradation. The function should return `True` if no cleavage sites are found. If a site is detected (`False`), it should return the problematic sequence.

Helpful links:
http://www.jbc.org/content/272/1/609.full.pdf
http://www.jbc.org/content/269/14/10790.long

Internal_RBS_Checker

Identifies potential internal ribosome binding sites (Shine-Dalgarno sequence + start codon) within the designed RNA that could lead to unintended translation initiation. The function should return `True` if no internal binding sites are found. If a site is detected (`False`), it should return the problematic sequence.

RNA_Interference_Checker

Checks for potential duplex formation between the native RNA and the designed RNA, aiming to prevent expression interference caused by reverse-complementary regions. The function should return `True` if no such duplexes are predicted. If a duplex is detected (`False`), it should return the problematic sequence and the name of the gene it interacts with.

Choose your own adventure

Identify and justify the biological relevance of the phenomenon being examined wrt TranscriptDesigner for *E. coli* expression

Write a checker and checker-test, etc. for it.

Highly recommended you run the idea by JCA before commitment

Existing Code

forbidden_sequence_checker.py

```python
class ForbiddenSequenceChecker:
    def __init__(self):
        self.forbidden = []

    def initiate(self):
        # Populate forbidden sequences
        self.forbidden = [
            "AAAAAAAA",  # poly(A)
            "TTTTTTTT",  # poly(T)
            "CCCCCCCC",  # poly(C)
            "GGGGGGGG",  # poly(G)
            "ATATATAT",  # poly(AT)
            "CAATTG",    # MfeI
            "GAATTC",    # EcoRI
            "GGATCC",    # BamHI
            "AGATCT",    # BglII
            "ACTAGT",    # SpeI
            "TCTAGA",    # XbaI
            "GGTCTC",    # BsaI
            "CGTCTC",    # BsmBI
            "CACCTGC",   # AarI
            "CTGCAG",    # PstI
            "CTCGAG",    # XhoI
            "GCGGCCGC",  # NotI
            "AAGCTT",    # HindIII
        ]
```

hairpin_checker.py

```python
# Iterate over the sequence in 50 bp chunks with 25 bp overlap
for i in range(0, len(dna) - chunk_size + 1, overlap):
    chunk = dna[i:i + chunk_size]

    # Get the count of hairpins and the hairpin string from hairpin_counter
    hairpin_count, hairpin_string = hairpin_counter(chunk, min_stem, min_loop, max_loop)

    # If more than 1 hairpin is found, return False and the problematic hairpin string
    if hairpin_count > 1:
        return False, hairpin_string

# If no problematic hairpin chunk is found, return True and None
return True, None
```

codon_checker.py

Checks for:

- **Codon Diversity:** Fraction of unique codons in the sequence.
- **Rare Codon Count:** Number of rare codons based on usage frequency.
- **Codon Adaptation Index (CAI):** Measures how adapted the codon usage is for optimal expression.

Thresholds:

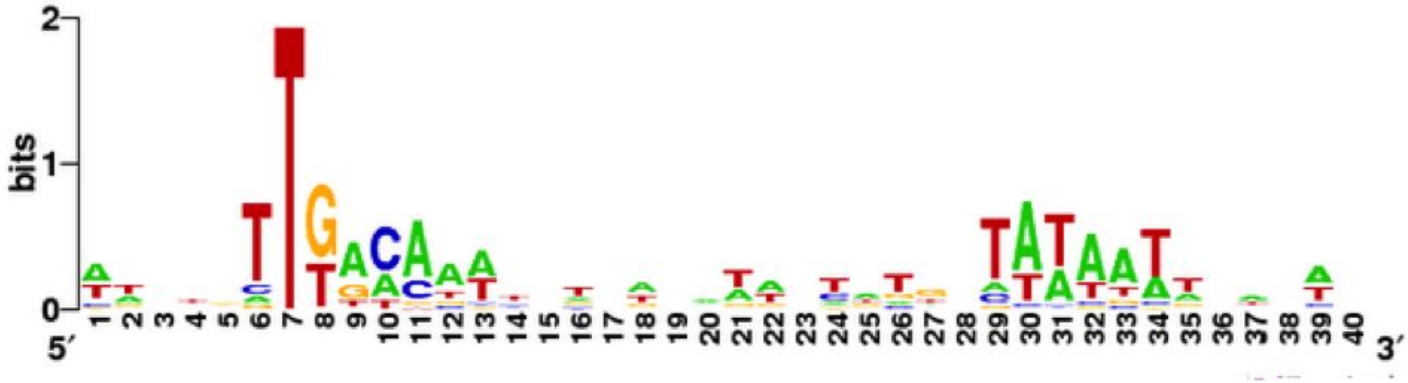
- **Diversity:** Must be ≥ 0.5
- **Rare Codons:** Must be ≤ 3
- **CAI:** Must be ≥ 0.2

Outcome:

- Returns a boolean if codons are "above board" based on these metrics.

internal_promoter_checker.py

- Identifies σ70 promoter regions in a DNA sequence
- Uses a position weight matrix (PWM) to identify the sequence pattern
- A PWM encodes the probability that a particular base is observed at a particular site within the sequence. For a test sequence, these probabilities are added to score the likelihood that a sequence is a σ70 promoter.
- The Checker Test uses J23119 promoter family variants to calibrate and assert correctness



internal_promoter_checker.py

```python
def initiate(self):
    """
    Initializes the Position Weight Matrix (PWM) based on a hardcoded Position Frequency Matrix (PFM).
    The PFM represents the nucleotide frequencies in 12 known sequences with constitutive promoters.
    This matrix is converted to a PWM, which is used to score other DNA sequences.
    """
    # Hard code a PFM matrix representing the nucleotide frequencies at 29 positions in known promoter sequences.
    pfm = [
        [0, 0, 0, 12, 0, 12, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 0, 12, 0, 12, 12, 0],  # A
        [0, 0, 0, 0, 12, 0, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0, 0],  # C
        [0, 0, 12, 0, 0, 0, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0, 0],  # G
        [12, 12, 0, 0, 0, 0, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 12, 0, 12, 0, 0, 12]   # T
    ]

    # Create an empty PWM matrix with the same dimensions as the PFM.
    ncols = len(pfm[0])
    self.pwm = [[0] * ncols for _ in range(4)]

    # Convert the PFM to a PWM by calculating the weight of each base at each position.
    for x in range(ncols):
        total = sum(pfm[y][x] for y in range(4))  # Total count for each position
        for y in range(4):
            prob_base = 0.25  # Assuming equal probability for each base in a random sequence
            freq = pfm[y][x]
            # Compute the log-odds weight for each nucleotide at each position
            w = (math.log((freq + math.sqrt(total) * prob_base) / (total + math.sqrt(total)) / prob_base)) / math.log(2)
            self.pwm[y][x] = w
```

internal_promoter_checker.py

```python
def run(self, seq):
    # Convert the sequence to uppercase and compute its reverse complement.
    seq = seq.upper()
    rc = reverse_complement(seq)
    combined = seq + "x" + rc  # Concatenate the original sequence and its reverse complement.

    sliding_frame = 29  # The sliding window size is 29 nucleotides.
    threshold = 9.134   # A threshold score for detecting promoter activity.

    # Slide over the sequence and calculate the score for each window.
    for i in range(len(combined) - sliding_frame + 1):
        score = 0.0
        partseq = combined[i:i + sliding_frame]
        for x, base in enumerate(partseq):
            # Map the base to its corresponding row in the PWM
            y = {'A': 0, 'C': 1, 'G': 2, 'T': 3}.get(base, -1)
            if y != -1:
                score += self.pwm[y][x]
        # If the score exceeds the threshold, the sequence likely contains a constitutive promoter.
        if score >= threshold:
            return False, partseq  # Promoter found, return the sequence
    return True, None  # No promoter detected in the sequence
```

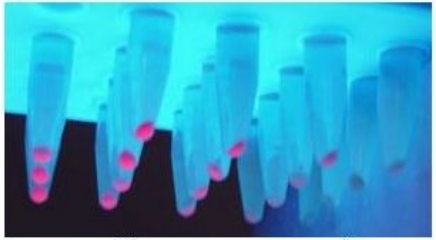
J23100 Promoter Family

BBa J23119 Main Page | Part Design | Physical DNA | Hard Information | Experience | Tools

Part:BBa_J23119

Designed by John Anderson Group: iGEM2006_Berkeley (2006-08-24)

constitutive promoter family member



```
                -35                 -10
                 |    10        20   |  30
                 ....|....|....|....|....|....|....|....|
J23119 (wt)      TTGACAGCTAGCTCAGTCCTAGGTATAATGCTAGC
J23100           TTGACGGCTAGCTCAGTCCTAGGTACAGTGCTAGC
J23101           TTTACAGCTAGCTCAGTCCTAGGTATTATGCTAGC
J23102           TTGACAGCTAGCTCAGTCCTAGGTACTGTGCTAGC
J23103           CTGATAGCTAGCTCAGTCCTAGGGATTATGCTAGC
J23104           TTGACAGCTAGCTCAGTCCTAGGTATTGTGCTAGC
J23105           TTTACGGCTAGCTCAGTCCTAGGTACTATGCTAGC
J23106           TTTACGGCTAGCTCAGTCCTAGGTATAGTGCTAGC
J23107           TTTACGGCTAGCTCAGCCCTAGGTATTATGCTAGC
J23108           CTGACAGCTAGCTCAGTCCTAGGTATAATGCTAGC
J23109           TTTACAGCTAGCTCAGTCCTAGGGACTGTGCTAGC
J23110           TTTACGGCTAGCTCAGTCCTAGGTACAATGCTAGC
J23111           TTGACGGCTAGCTCAGTCCTAGGTATAGTGCTAGC
J23112           CTGATAGCTAGCTCAGTCCTAGGGATTATGCTAGC
J23113           CTGATGGCTAGCTCAGTCCTAGGGATTATGCTAGC
J23114           TTTATGGCTAGCTCAGTCCTAGGTACAATGCTAGC
J23115           TTTATAGCTAGCTCAGCCCTTGGTACAATGCTAGC
J23116           TTGACAGCTAGCTCAGTCCTAGGGACTATGCTAGC
J23117           TTGACAGCTAGCTCAGTCCTAGGGATTGTGCTAGC
J23118           TTGACGGCTAGCTCAGTCCTAGGTATTGTGCTAGC
```

Variant	RFP (au)
J23112	1
J23103	17
J23113	21
J23109	106
J23117	162
J23114	256
J23115	387
J23116	396
J23105	623
J23110	844
J23107	908
J23106	1185
J23108	1303
J23118	1429
J23111	1487
J23101	1791
J23104	1831
J23102	2179
J23100	2547

http://parts.igem.org/Part:BBa_J23100

Strategy Discussion

Questions?